

Entrega Guía 1: Gestión de tareas

April 25, 2026

Link al repositorio con el código completo: https://github.com/Emi-P/sistemas_de_tiempo_real

1 Ejercicio 1

1.1 Implementación

Las tareas implementadas son todas de esta pinta, con la diferencia del delay utilizado para setear la variable Delay_ms y los LEDs utilizados son uno distinto para cada tarea.

```
void vTareaParpadeo200(void *pvParameters){
const TickType_t Delay_ms = pdMS_TO_TICKS( 200 );
GPIO_TypeDef* port = LD4_GPIO_Port;
uint16_t pin = LD4_Pin;
while (1){
    HAL_GPIO_TogglePin(port, pin);
    HAL_Delay( Delay_ms );
}
}
```

Y se crean las tareas de esta manera

```
xTaskCreate(
    vTareaParpadeo200,
    "Blink200",
    configMINIMAL_STACK_SIZE,
    NULL,
    tskIDLE_PRIORITY,
    NULL
);
... // 400ms, 600ms

xTaskCreate(
    vTareaParpadeo800,
    "Blink200",
    configMINIMAL_STACK_SIZE,
    NULL,
    tskIDLE_PRIORITY,
    NULL
);
```

1.2 Conclusiones

Todas las tareas se ejecutan. Esto es obvio pues cada tarea tiene asignado un LED distinto y estos están funcionando a periodos diferentes.

Pareciera que hay concurrencia pero dado que el procesador es de un solo núcleo, sabemos que es una ilusión provocada por el scheduling de FREERTOS. Mas claramente comprobamos en clase usando el osciloscopio, hay una medible diferencia entre el inicio de una tarea a pesar de que esto es aparentemente en el mismo instante.

Los tiempos de conmutación no serán precisos. Pues dado que las esperas son no-bloqueantes. El scheduler tendrá que intervenir para repartir la cpu y en dichos cambios de contexto se introduce jitter.

2 Ejercicio 2

2.1 Implementación

La única tarea implementada es

```
void vTareaParpadeo(void *pvParameters){
    // Recibir parametros
    struct ParpadeoParameters parameters = \
        *(struct ParpadeoParameters *) pvParameters;

    const TickType_t xDelays = pdMS_TO_TICKS( parameters.ms );
    while (1){
        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);
        vTaskDelay( xDelays );
    }
}
```

Y se inician en main, las tareas:

```
// Crear las configuraciones para cada tarea
static struct ParpadeoParameters task200Parameters = {
    .Pin = LD4_Pin,
    .Port = LD4_GPIO_Port,
    .ms = 200
};

// ... 400ms, 600ms, y LEDs 5 y 3
static struct ParpadeoParameters task800Parameters = {
    .Pin = LD6_Pin,
    .Port = LD6_GPIO_Port,
    .ms = 800
};

xTaskCreate(
    vTareaParpadeo,
    "Blink200",
    configMINIMAL_STACK_SIZE,
    &task200Parameters,
    tskIDLE_PRIORITY,
    NULL
);

// ... 400ms, 600ms
xTaskCreate(
    vTareaParpadeo,
    "Blink800",
    configMINIMAL_STACK_SIZE,
    &task800Parameters,
    tskIDLE_PRIORITY,
    NULL
);
```

2.2 Conclusiones

En el Desafío 2 las tareas utilizan `vTaskDelay`, lo que introduce estados de bloqueo (Blocked). A diferencia del Desafío 1, donde las tareas permanecían activas usando busy-wait, ahora ceden el CPU voluntariamente. Esto permite al scheduler gestionar mejor la ejecución, reduce la competencia por CPU y disminuye el jitter. Como resultado, los períodos de conmutación son más estables, aunque siguen limitados por la resolución del tick del sistema.

3 Ejercicio 3

3.1 Implementación

```
void vHandlePush(void *pvParameters) {
    while(1){
        if ( HAL_GPIO_ReadPin(B1_GPIO_Port , B1_Pin) ==
GPIO_PIN_SET ){
            HAL_GPIO_WritePin(LD6_GPIO_Port , LD6_Pin ,
GPIO_PIN_SET);
        }
        else if ( HAL_GPIO_ReadPin(B1_GPIO_Port , B1_Pin) ==
GPIO_PIN_RESET ){
            HAL_GPIO_WritePin(LD6_GPIO_Port , LD6_Pin ,
GPIO_PIN_RESET);
        }
        vTaskDelay(pdMS_TO_TICKS(10)); // Bloquearse 10ms
    }
}

void vTareaParpadeo(void *pvParameters){
    // Recibir parametros
    struct ParpadeoParameters parameters = *(struct
ParpadeoParameters *) pvParameters;
    while (1){
        HAL_GPIO_TogglePin(parameters.Port , parameters.Pin);
        HAL_Delay( 500 );
    }
}
```

`vHandlePush` hará polling cada 10ms para verificar el estado del botón y aplicar el estado correspondiente al LED. La otra tarea se reutilizó de ejercicios pero ahora es no-bloqueante por usar `HAL_Delay` como servicio de espera.

Se crean las tareas de esta manera

```
static struct ParpadeoParameters LedBloqueante500ms = {
    .Pin = LD4_Pin,
    .Port = LD4_GPIO_Port,
    .ms = 500
};

xTaskCreate(
    vTareaParpadeo,
    "LedBloqueante500ms",
    configMINIMAL_STACK_SIZE,
    &LedBloqueante500ms,
    2,
    NULL
);
xTaskCreate(
    vHandlePush,
    "PollingBoton",
    configMINIMAL_STACK_SIZE,
    NULL,
    3,
    NULL
);
```

3.2 Conclusiones

Al usar tiempos de `vTaskDelay` mas altos, el botón responde muy tarde al pulsar y soltar. Y es posible hallar ventanas donde se puede presionar y soltar sin respuesta de ningún tipo. Un `vTaskDelay` de 10ms satisface la consigna, pues asegura que la tarea no estaría más de 10ms bloqueada y sabemos que una vez se ponga `READY`, el scheduler decide ejecutarla. Esto no se asegura con total precision, pues se introduce un jitter por el cambio de contexto, dado que la otra tarea siempre está ejecutando el resto del tiempo.

4 Ejercicio 4

4.1 Implementación

```
void vTareaParpadeoA(void *pvParameters){
    struct ParpadeoParameters parameters = *(struct
ParpadeoParameters *) pvParameters;
    while (1){
        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);
        HAL_Delay( 100 ); // Trabajo simulado
        vTaskDelay(pdMS_TO_TICKS(parameters.ms)); // Handlear
cada 10ms
    }
}

void vTareaParpadeoB(void *pvParameters){
    struct ParpadeoParameters parameters = *(struct
ParpadeoParameters *) pvParameters;
    TickType_t xLastWakeTime;
    xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(parameters.ms);
    while (1){
        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);
        HAL_Delay( 100 ); // Trabajo simulado
        vTaskDelayUntil(&xLastWakeTime, xFrequency);
    }
}
```

Notar que la TareaB utiliza vTaskDelayUntil en vez de vTaskDelay como en TareaA.

Las tareas se crean con la misma prioridad, y usando LED's diferentes. Notar que se setea el campo ms de forma diferente

```
static struct ParpadeoParameters paramsTareaA = {
    .Pin = LD4_Pin,
    .Port = LD4_GPIO_Port,
    .ms = 400
};
static struct ParpadeoParameters paramsTareaB = {
    .Pin = LD5_Pin,
    .Port = LD5_GPIO_Port,
    .ms = 500
};

xTaskCreate(
    vTareaParpadeoA,
    "LedBloqueante-Delay500ms",
    configMINIMAL_STACK_SIZE,
    &paramsTareaA,
    0,
    NULL
);
xTaskCreate(
    vTareaParpadeoB,
    "LedBloqueante-DelayUntil500ms",
    configMINIMAL_STACK_SIZE,
    &paramsTareaB,
    0,
    NULL
);
```

4.2 Conclusiones

El tiempo utilizado en la TareaA es de 400ms, pues queremos mantener una periodicidad de 500ms sabiendo que se realiza trabajo de 100ms en el medio. En el caso de TareaB, vTaskDelayUntil es capaz de compensar el retardo agregado por el trabajo de 100ms pues considera el tiempo absoluto desde que inicio la tarea.

Si ambos argumentos son iguales de 500ms, se observa un desfase muy exagerado en la conmutación de los LED's. Dado que la TareaA termina el trabajo de 100ms y es entonces que comienza a esperar 500ms para desbloquearse. En cambio la TareaB considera la diferencia.

5 Ejercicio 5

5.1 Implementación

```
void vTareaParpadeoA(void *pvParameters){
    struct ParpadeoParameters parameters = *(struct
    ParpadeoParameters *) pvParameters;

    UBaseType_t prioOriginal = uxTaskPriorityGet(NULL);
    TickType_t tiempoFin = 0;
    BaseType_t enBoost = pdFALSE;

    while (1){
        if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) ==
        GPIO_PIN_SET){
            vTaskPrioritySet(NULL, prioOriginal + 1);
            tiempoFin = xTaskGetTickCount() + pdMS_TO_TICKS
(3000);
            enBoost = pdTRUE;
        }

        if (enBoost == pdTRUE && xTaskGetTickCount() >=
tiempoFin){
            vTaskPrioritySet(NULL, prioOriginal);
            enBoost = pdFALSE;
        }

        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);

        HAL_Delay(parameters.ms);
    }
}

void vTareaParpadeoB(void *pvParameters){
    struct ParpadeoParameters parameters = *(struct
    ParpadeoParameters *) pvParameters;
    TickType_t xLastWakeTime;
    xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(parameters.ms);
    while (1){
        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);
        HAL_Delay(xFrequency);
    }
}
```

La lógica de la TareaB es sencilla. Solamente se encarga de hacer un toggleo periodico del LED. Notar que a pesar de tener intención de considerar Delays absolutos, no va a poder conservar la periodicidad si la TareaA sube de prioridad.

La TareaA agrega algo de lógica para mantener el parpadeo aunque se esté atravesando los 3 segundos de boosteo de prioridad. Al detectar que se presiona el botón, setea en pdTRUE la variable que indica que se encuentra durante un boosteo. Además tiempoFin guarda el instante de tiempo en que se debería volver a la prioridad original. Si ya pasó el momento tiempoFin y se esta un boosteando se vuelve la variable enBoost a pdFALSE.

Las esperas en ambas tareas son, no-bloqueantes para observar el starving al tener la TareaA en una prioridad mayor.

5.2 Conclusiones

Al restablecerse la prioridad de la TareaA, la conmutación del LED para la TareaB vuelve a ser normal

El conteo de los 3 segundos es mediante `xTaskGetTickCount()`, de forma que se calcula el tiempo en el que pasaran los 3 segundos. Podría haberse utilizado una espera de 3 segundos usando `HAL_Delay()`, pero con esa implementación no se mantendría el parpadeo de la TareaA. No se puede utilizar `vTaskDelay` ni `vTaskDelayUntil`, pues bloquearían el proceso de la TareaA resultando que la TareaB pueda reanudar en esas ventanas de tiempo a pesar de ser de menor prioridad.

6 Ejercicio 6

6.1 Implementación

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    if (GPIO_Pin == GPIO_PIN_0){
        xSemaphoreGiveFromISR(xButtonSemaphore, &
xHigherPriorityTaskWoken);

        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}
void vTareaBoton(void *pvParameters){
    while (1){

        xSemaphoreTake(xButtonSemaphore, portMAX_DELAY);

        HAL_GPIO_TogglePin(LD3_GPIO_Port, LD3_Pin);
        vTaskDelay(pdMS_TO_TICKS(50)); // Debounce

    }
}
```

El código del callback es sencillo, habilita la posibilidad de tomar el semáforo cada vez que se levanta la interrupción si fue por haberse presionado el botón en GPIO_PIN_0.

`vTareaBoton` está casi siempre bloqueada, pues cada vez que ejecuta `xSemaphoreTake`, debe esperar a que el callback le de la posibilidad de tomarlo nuevamente.

Agrega una pequeña espera para reducir el bounce del botón.

6.2 Conclusiones

Se propone hacer que el LED a encender rote entre los 4 LEDs cada vez que se ejecuta el bucle principal de la tarea.

De esta forma si la tarea estuviera haciendo polling en vez de actuar únicamente por ocurrir una interrupción, sería impredecible el próximo LED que se enciende. Dado que se usa este sistema que bloquea la tarea hasta que se toca el botón será predecible saber cual es el próximo LED que se enciende al presionar el botón.

Se implementa de esta forma:

```
void vTareaBoton(void *pvParameters) {
    uint16_t leds[] = {LD3_Pin, LD4_Pin, LD5_Pin, LD6_Pin};
    int i = 0;

    while (1) {
        xSemaphoreTake(xButtonSemaphore, portMAX_DELAY);
        HAL_GPIO_WritePin(GPIOD, leds[i], GPIO_PIN_RESET);
        i = (i + 1) % 4;
        HAL_GPIO_WritePin(GPIOD, leds[i], GPIO_PIN_SET);
        vTaskDelay(pdMS_TO_TICKS(50)); // Debounce
    }
}
```